

Fashion AI: Project Report

Wardrobe digitisation and outfit recommendation with zero-shot garment detection, CLIP embeddings, a learned compatibility signal and a validation-fitted ranking model.

Report date: 2026-10-04. All numbers below were produced by the code in this repository on the hardware stated in section 23. "Verified" means the result was re-run before this report was written; earlier single measurements are labelled as such.

1. Executive Summary

Fashion AI is a local Streamlit application. A user photographs their clothes; the app detects garments with YOLO-World (zero-shot), shows the uncertain detections for human review, stores the confirmed garments with CLIP embeddings in a local SQLite wardrobe, and builds ranked, explained outfits from the *whole* eligible wardrobe.

The final system is the result of several rounds of repair. The original code base did not start, had incompatible detector APIs, loaded CLIP three different ways and reported "benchmark" numbers produced from random vectors. Two later failures shaped the final architecture:

1. **A bad detector label became a confidently recommended, impossible outfit.** On the Ultralytics `bus.jpg` sample, a man in a coat and jeans was stored as "jumpsuit 0.72" and "jumpsuit + shoes" was ranked as a strong outfit (~0.80).
2. **Only 5–7 wardrobe items were effectively used** because of a chain of caps and cutoffs in detection, review and generation.

The final pipeline keeps detector uncertainty (alternatives, flags, a *needs review* state), rejects structurally impossible outfits *before* any scoring, enumerates every valid combination, ranks with a combiner whose weights were fitted on validation data, applies a validation-chosen quality threshold and selects diverse looks without padding.

Key quantitative results (Polyvore-derived, item-disjoint test split; details in section 22):

Scorer	Pair ROC-AUC (41,745 test pairs)
Random	0.503
CLIP cosine similarity	0.580
Hand-written app heuristic	0.580
Learned compatibility model	0.725 (95% CI 0.720–0.730)
Fitted ranker, with learned model	0.727
Fitted ranker, no learned checkpoint	0.604

ROC-AUC 0.725 is **not** "72.5% accuracy". It is the probability that an observed Polyvore pair outranks a constructed negative pair on this specific evaluation. It is not a measure of fashion quality, taste, or performance on users' photos.

An empirical finding worth stating up front: **the hand-written colour rules did not predict real outfits**. When the ranker was fitted, colour received a negative weight, so colour is still computed and shown but is not used to rank.

2. Project Objective

Build an application that:

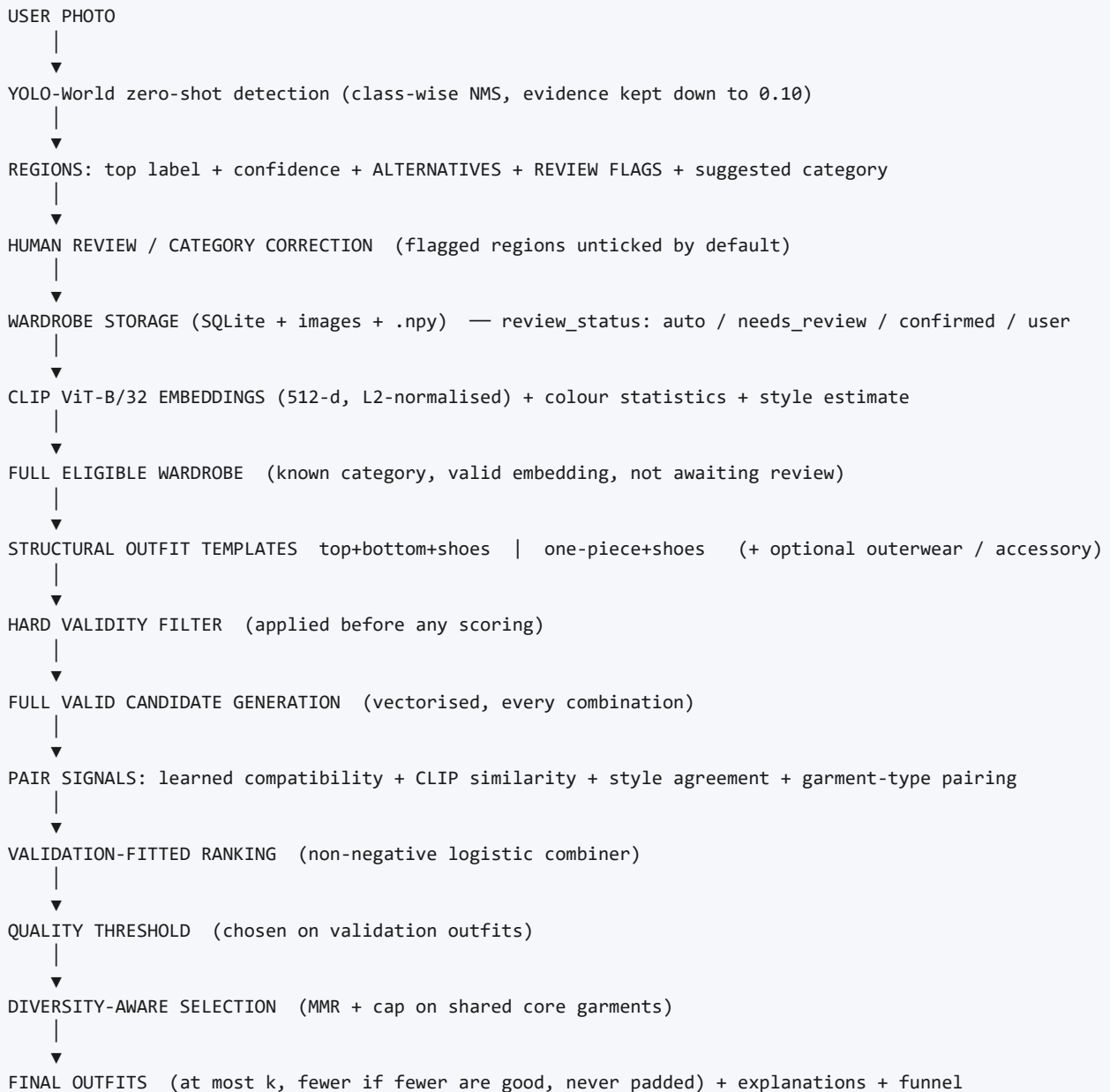
- turns photos of a user's clothes into a structured digital wardrobe;
- recommends complete, wearable outfits from that wardrobe;
- explains each recommendation using only values it actually computed;
- is honest about uncertainty: uncertain detections are surfaced, not hidden, and nothing (accuracy, attributes, benchmarks) is fabricated;
- uses a learned compatibility signal only when it is trained on real data and demonstrably beats simple baselines;
- runs locally, with CUDA as the recommended runtime and a working CPU fallback.

3. Original Problems

Area	Problem found
Entry point	The Flask app crashed at import without a gitignored checkpoint, demo images and embeddings.
Detector	Two incompatible APIs (detect vs detect_and_crop, path vs box).
Imports	<code>src.ml.model</code> imports that did not exist.
Embeddings	Three separate CLIP loaders (app, training, legacy scripts).
Training	<code>train_compatibility.py</code> trained on random Gaussian vectors with <code>BCELoss</code> applied to logits and reported the result as a benchmark. Meaningless by construction.
Storage	Uploads written to a shared JSON file; no integrity checks.
Attributes	A SAM "mock" fallback produced fake 3×3 grid crops; LLaVA attributes (material, pattern) were guesses presented as facts.
Recommendation	No hard structural rules: a dress could be paired with trousers; a single detector label was trusted completely.
Coverage	Per-slot caps, top-N truncation and repeat padding meant only ~5–7 items were effectively used.

The archived code is in `experiments/` (see `experiments/README.md`); the synthetic-vector trainer was deleted and remains only in git history.

4. Final System Architecture



Why it is built this way. The earlier failure chain was

bad detector label → wrong category → structurally invalid outfit → high compatibility score → bad recommendation

A compatibility model trained on product pairs has no notion of whether a combination is wearable; it will happily score "jumpsuit + shoes" highly if the crops look coherent. Each layer of the new architecture breaks one link:

- uncertainty is kept and surfaced (alternatives, flags), so a wrong label is visible instead of silently trusted;
- uncertain items are excluded until a person reviews them;
- structural validity is a hard gate *before* scoring, so no score can rescue an impossible outfit;
- the learned model is one signal in a fitted ranking, not the decision;

- a validation-chosen threshold decides what is good enough to show, so weak combinations are not used to fill the requested count.

Concern	Module
Configuration	<code>src/config.py</code> (env vars / <code>.env</code> ; the only place model names, paths and thresholds are defined)
Image loading	<code>src/imaging.py</code>
Detection	<code>src/detection/detector.py</code> , <code>schemas.py</code> , <code>categories.py</code>
Embeddings	<code>src/embeddings/clip_encoder.py</code> (the only CLIP loader)
Attributes	<code>src/attributes/colors.py</code> , <code>style.py</code>
Storage	<code>src/wardrobe/store.py</code> , <code>manager.py</code> , <code>schemas.py</code>
Pair signals + fitted ranker	<code>src/recommendation/pair_model.py</code> , <code>ranker_model.json</code>
Validity rules	<code>src/recommendation/validity.py</code>
Generation, diversity, explanations	<code>src/recommendation/outfit_generator.py</code>
Learned model + training	<code>src/models/</code> , <code>src/training/</code>
Wiring	<code>src/service.py</code>
UI	<code>app.py</code> , <code>src/ui/</code>

5. Computer Vision / Detection Pipeline

- **Model.** Ultralytics YOLO-World `yo1ov8s-worldv2.pt`, prompted with 26 clothing words mapped to six outfit slots (top, bottom, one_piece, outerwear, shoes, accessory). It is a zero-shot, open-vocabulary detector; it was **not** trained or fine-tuned on fashion data in this project.
- **Vocabulary cache.** The text prompts are encoded once and saved as `checkpoints/yo1ov8s-worldv2.fashion-<hash>.pt`; later starts load it in $\sim 0.1\text{--}0.4$ s without the CLIP text encoder.
- **Inference.** On a copy at most 2048 px on the long side; boxes are mapped back to original coordinates and crops are cut from the full-resolution image.
- **Validation of every prediction.** Known label, finite confidence in $[0, 1]$, box clipped to the image, ≥ 16 px per side, $\geq 0.2\%$ of the image area. Malformed predictions are skipped and counted, never raised.
- **Class-wise NMS.** YOLO-World scores each prompt independently. Agnostic NMS (the old setting) kept only the single highest label per area and discarded competing evidence. The detector now runs class-wise NMS and keeps evidence down to 0.10.
- **Region grouping.** Overlapping boxes ($\text{IoU} \geq 0.55$) are grouped into one region. The highest-scoring label is the region label; the others become `alternatives` (label, slot, confidence). Regions below `DETECTION_FLOOR` (0.15) are not shown.
- **Deterministic ids** (hash of image, label and box) and a rebuilt crop folder on re-detection.

On `bus.jpg`, class-wise detection with the 0.10 evidence floor yields 18 raw predictions grouped into 10 regions; the old agnostic/0.25 setting produced 7.

6. Detection Uncertainty and Human Review

A region is **flagged** when:

Flag	Rule
low_confidence	score below CONFIDENCE_THRESHOLD (0.25)
ambiguous_category	an alternative from a <i>different</i> slot scores $\geq 0.6 \times$ the top score
contains_separates	a dress/jumpsuit region that the detector also labelled as top/bottom/outerwear, or that contains a separate garment box ($\geq 80\%$ inside it and $\leq 75\%$ of its area); the suggested category is that alternative's slot
small_crop	crop smaller than 32 px on a side

Review workflow.

1. The review form lists every region with its crop, score, alternatives ("Also detected as: coat (Outerwear) 0.20, jacket (Outerwear) 0.13"), the flag messages and a suggested category.
2. Flagged regions are **unticked** by default; when the region has a suggestion, the category selector is preselected to it.
3. Ticking a flagged region and adding it counts as a review: the item is stored as `confirmed` with the chosen category.
4. Flagged detections stored without review (for example via `scripts/ingest.py`) are stored as `needs_review`. The gallery shows a warning with the flags and alternatives, a "Category is correct" button and an edit panel preselected to the suggestion.
5. **Items awaiting review are excluded from outfit generation and recommendations**; the outfit page lists each excluded item with its reason.

Regression example: `bus.jpg` (real YOLO-World, verified).

Before: coat + jeans → "jumpsuit 0.72" (alternatives discarded) → one-piece + shoes accepted → learned score ~ 0.80 → recommended outfit.

After: "jumpsuit 0.72", alternatives coat 0.20 / jacket 0.13, flag `contains_separates`, suggested *outerwear* → unticked in the review form → if added without review it is `needs_review` and excluded. With both sample photos ingested unreviewed, the generator reports "*Not enough items for an outfit: add at least one top and one bottom ... Usable items: 2 bottom, 3 outerwear, 2 shoes*" instead of inventing an outfit. On `zidane.jpg` the "jumpsuit 0.40" region is flagged `ambiguous_category` + `contains_separates` (alternatives jacket 0.29, shirt 0.24, blazer 0.24).

Not claimed. The flags catch the patterns they encode. A confidently wrong label with no competing evidence (for example a single clean "dress" box on a long coat) is not flagged. The detector remains imperfect: pants/jeans confusion and missed small accessories were observed.

7. Wardrobe Data Model

DATA_DIR/	(default data/, gitignored)
wardrobe/wardrobe.db	SQLite: id, category, image_path, embedding_path, confidence,
created_at, metadata.json	
wardrobe/images/<id>.jpg	stored garment image (longest side ≤ 768 px)
wardrobe/embeddings/<id>.npz	float32 unit-norm vector (loaded with <code>allow_pickle=False</code>)
uploads/	original uploads
processed/crops/	detector crops (rebuilt on re-detection)

- Paths are stored relative to `DATA_DIR`.

- Files are written to temporary names and renamed; CLIP runs before anything is written, so a failed embedding leaves no files; a failed insert removes what it wrote.
- `metadata_json` holds the detector label, label source, bbox, source image, dominant colours, style scores, `alternatives`, `review_flags`, `suggested_category` and `review_status`.
- Every item records `embedding_model` and `embedding_dim`; a model change marks embeddings **stale** and excludes them instead of mixing spaces. Diagnostics can re-embed from the stored images.
- Re-adding the same image (cosine ≥ 0.97 , same category) is flagged as a possible duplicate; data is never dropped silently.
- Storage has no row limit; the gallery paginates for display only.

8. CLIP Embedding System

- Hugging Face `openai/clip-vit-base-patch32`, 512-d image embeddings, L2-normalised; the dimension is read from the model config.
- One loader (`src/embeddings/clip_encoder.py`), cached and lazy, shared by the app, the dataset embedding script and the training/evaluation code, so wardrobe items and training data live in the same space.
- Used for: the learned model's input, the CLIP-similarity signal, the zero-shot style estimate (softmax over 7 style prompts) and duplicate detection.
- Outfit generation and recommendations use stored embeddings and never load CLIP. The checkpoint's encoder compatibility is checked from the CLIP config without loading the model.

9. Outfit Candidate Generation

The previous 5–7 item limitation. Root causes, in pipeline order:

1. *Cross-label merging*: agnostic NMS kept one label per area, so fewer garments were detected per photo.
2. *Score cutoff*: detections below 0.25 were dropped.
3. *Review threshold*: the review form unticked everything below 0.3.
4. *Per-slot candidate cap*: generation pruned each slot to a few members.
5. *Top-N base outfits*: optional layers were tried only on the top 20–30 base outfits.
6. *Layer candidate cap*: at most 12 outerwear/accessory candidates.
7. *Repeat padding*: a repeat cap plus padding filled the requested count with near-copies.

The SQLite store never had a LIMIT; the losses were all in detection, review and generation. All seven were removed or replaced.

Current generation.

- Input: **all eligible items**.
- Pair matrices for all eligible pairs are computed in one batched pass.
- Templates `top + bottom + shoes` and `one_piece + shoes` are enumerated with numpy (`meshgrid`) over the full slot pools.
- The only size limit is a **compute guard of 200,000 base combinations per template**. If exceeded, slot pools are pruned by each item's best pair score and the UI reports it. A wardrobe of 50 tops $\times$ 30 bottoms $\times$ 20 shoes is 30,000 combinations.
- Optional layers: for each candidate, the best non-conflicting outerwear and accessory is attached only if it does not lower the outfit score (a must-include layer is forced).

- The user chooses 1–20 looks. If fewer strong, distinct looks exist, fewer are returned with a message; the list is **never padded**.

Validated 29-item example (real product photos). A wardrobe of 8 tops, 6 bottoms, 5 shoes, 4 outerwear, 3 dresses and 3 accessories, drawn deterministically from the Polyvore *test* split by `scripts/demo_wardrobe.py` (none of these items were used to train the model or fit the ranker), with the learned model enabled:

Looks requested	Returned	Unique items used
3	3	11
5	5	19
10	10	25
20	20	28

All 29 items eligible; **255 core combinations** ($8 \cdot 6 \cdot 5 + 3 \cdot 5$), **5,100** counting optional-layer variants; 0 rejected by validity; 0 pruned; **148 above the quality threshold**. Identical on CPU and CUDA.

10. Hard Semantic Validity Layer

`src/recommendation/validity.py`, applied **before** ranking. A rejected combination is never scored, shown or rescued by a model score.

- The body is exactly top + bottom **or** one dress/jumpsuit.
- A one-piece is never combined with a separate top or bottom.
- At most one item per slot (no two tops, two bottoms, two pairs of shoes).
- Shoes are required.
- Outerwear and accessories are optional layers and never substitute for the body.
- Every item must be eligible (known category, valid embedding, not awaiting review).
- Two items cut from the same box of the same photo ($\text{IoU} \geq 0.5$) or with near-identical images ($\text{CLIP cosine} \geq 0.97$) are one garment. Containment alone is **not** treated as "same garment": a coat box legitimately encloses the shirt beneath it.

The test suite contains a curated adversarial set (one-piece + top, one-piece + bottom, two tops, two pairs of shoes, no shoes, top without bottom, bottom without top, outerwear standing in for a top, accessory + shoes, layers with no body) plus the same-region case; each must be rejected with an explanatory reason.

11. Compatibility Model

- `OutfitCompatibilityNet`: an MLP over $[a, b, |a-b|, a \odot b]$ of two CLIP embeddings (hidden $1024 \rightarrow 256 \rightarrow 64$, dropout 0.3), outputting a logit.
- Trained with `BCEWithLogitsLoss`; inference applies sigmoid and averages (a, b) and (b, a) , so it is symmetric.
- Checkpoint format `fashion-ai/compatibility/v1`: weights, model config, encoder config, metrics, dataset gate verdict, dataset version.
- **Used automatically only if** the checkpoint's encoder matches `CLIP_MODEL` **and** it beat the strongest baseline on its test split (95% CI of the paired ROC-AUC difference above 0).
`USE_LEARNED_MODEL=true|false` overrides.

- It is a **learned compatibility signal**: it estimates whether two products co-occurred in curated Polyvore sets. It does not understand fashion and is not a universal judge of compatibility. Its scores are uncalibrated (ECE 0.11).
- The checkpoint is generated locally and **not committed**.

12. Ranking System

Pair signals (src/recommendation/pair_model.py):

Feature	Definition	Used for ranking
learned_logit	logit of the learned model (only if one is in use)	yes
clip_cos	CLIP image-image cosine	yes
style_sim	Bhattacharyya agreement of CLIP style distributions	yes
category	slot-pair plausibility table (top+bottom 1.0, ...)	yes
colour	relation of the two primary colours (rules below)	no : shown only

Fitting. scripts/fit_ranker.py rebuilds the audited, item-disjoint pair splits and computes the features exactly as the app does. It fits a logistic regression on **validation** pairs (observed pairs vs constructed random and hard negatives), in two variants (with and without learned_logit). Weights are constrained to be non-negative: every feature is defined so that higher means more compatible, so a negative weight means the data contradicts the feature; that feature is dropped and the model refitted.

	with learned model	without (fresh clone)
Standardised weights	learned 0.924, CLIP 0.011, style 0.109, category 0.183	CLIP 0.157, style 0.261, category 0.133
Dropped	colour (-0.100)	colour (-0.219)

Outfit score = weighted mean of the pair scores (pairs involving an accessory weigh 0.5). A requested style/occasion adds $\text{WEIGHT_CONTEXT} \times (\text{fit} - 0.5)$ to the rank only (default weight 0.10).

The colour finding. The colour rules score relations between primary colours: neutral + contrasting neutral 0.80, accent + neutral 0.75, monochrome with lightness contrast 0.70, analogous 0.70, complementary 0.60, low-contrast neutrals (e.g. black + navy) 0.55, tone-on-tone 0.50, conflicting hues 0.35. They encode common styling advice that favours contrast. On Polyvore validation pairs the fitted weight was negative in both variants: observed outfits are frequently tonal (same or neighbouring colours), which the rules score low. Rather than keep an intuitive but data-contradicted signal, colour is:

- still computed (dominant colours by k-means + HSV naming);
- still visible (gallery swatches, score details labelled "*shown for information; not used in ranking*", the outfit page and the Diagnostics page);
- **not used in ranking**.

This finding is specific to this dataset and these rules; it does not show that colour is irrelevant to outfits, only that these rules do not predict Polyvore co-occurrence.

Explanations. ✓ reasons are the structure plus features whose contribution to the fitted logit is $\geq +0.15$; ⚠ caveats are contributions ≤ -0.15 , weak style/occasion fit and items detected with moderate confidence (< 0.40 , unless confirmed). Silhouette, fit, material and pattern are not computed and are never mentioned (tested).

13. Diversity System

Greedy maximal-marginal-relevance selection over the strong candidates (the top 5,000 by rank):

- next look = $\text{argmax of rank} - 0.6 \times (\text{item overlap with any chosen look})$;
- hard cap: a look may share at most half of its core garments with any chosen look;
- near-duplicate copies of a garment count as the same garment, so two looks never differ only by which copy is used.

Diversity is a ranking objective, not a quota: popular items can recur (e.g. one pair of jeans in 3 of 10 looks) while the cap prevents near-identical lists. Per-item exposure (appearances among strong candidates vs in the returned looks) is shown in the UI.

14. Quality Threshold

Chosen on **validation** outfits: 1,238 real Polyvore outfits vs the same number of category-preserving fakes (each item swapped for a random item of the same slot), threshold = $\text{argmax}(\text{TPR} - \text{FPR})$.

	with learned model	without
Threshold	0.517	0.516
Real test outfits passing (TPR)	71%	59%
Random same-category swaps passing (FPR)	16%	34%
Outfit ROC-AUC, real vs swaps (2,594 test outfits)	0.862	0.664

Random same-category swaps are easy negatives; these numbers mean "real outfits tend to pass and random ones tend not to", not a measure of quality. On users' photos the threshold is a conservative cut-off, not a guarantee.

15. Dataset

Maryland Polyvore (Han, Wu, Jiang, Davis. *Learning Fashion Compatibility with Bidirectional LSTMs*. ACM Multimedia 2017).

- Outfits and the official split: [xthan/polyvore-dataset](#) (Apache-2.0), pinned to commit [ba8aa71b6589](#).
- Item images: the ungated Hugging Face mirror [Marqo/polyvore](#) (Apache-2.0), pinned to revision [8c782ee447fa](#).
- The product images originate from Polyvore.com and may be third-party copyright: **research use, no redistribution**. No dataset images are committed to this repository.
- 18,536 outfits (14,704 / 1,238 / 2,594 train/valid/test), 71,848 items after preparation, 6 slots.

What the data is and is not:

- Polyvore sets are **curated product collages** made by one online community; they do not represent real-world fashion in general.
- **Labels are noisy** (e.g. an item categorised "tank tops" whose title describes sunglasses appeared in the demo wardrobe).
- Positives are **co-occurrence**, not human-judged compatibility or preference.

- **Product imagery** (clean, white background) differs from the rectangular crops the app makes from users' photos.
- Constructed negatives mean "not observed together", not "judged incompatible"; random negatives are relatively easy.

The better-known Polyvore Outfits release (Vasileva et al., ECCV 2018) is gated (login, institutional approval) and is not downloaded automatically.

16. Dataset Acquisition

`scripts/download_dataset.py`:

- both sources pinned to exact revisions; every file's size and SHA-256 are verified (`artifacts/dataset_manifest.json` records URLs, sizes, hashes);
- HTTP Range resume of partial downloads (tested with a local server);
- safe archive extraction (rejects absolute paths, `..` and links; tested);
- preparation keeps only items whose fine category maps unambiguously to a slot (decor, beauty, tech, swimwear and generic "Clothing" are dropped, with counts recorded); outfits with fewer than 2 usable items are dropped;
- 2.5 GB download, ~6 GB on disk after preparation;
- the app itself never downloads training data.

17. Dataset Audit

`src/training/audit.py` (also `scripts/audit_dataset.py`) reports item and outfit counts, category balance, outfit sizes, metadata/text availability, missing/corrupt images, exact and perceptual (dHash) duplicates, cross-split leakage and effective size after cleaning and item-disjoint splitting, then classifies the dataset as SUFFICIENT / SUFFICIENT WITH LIMITATIONS / INSUFFICIENT. Training refuses INSUFFICIENT data. Thresholds ($\geq 2,000$ training outfits, $\geq 5,000$ items, $\geq 1,000$ test positives, core categories present, $\leq 5\%$ missing/corrupt) are documented engineering judgements.

Verdict for this dataset: SUFFICIENT WITH LIMITATIONS (`artifacts/dataset_audit.md`):

- 0 missing, 0 corrupt images out of 71,848; 84 images under 64 px; duplicate-image rate 0.8%;
- 130 duplicate-image groups spanned different splits and were collapsed during cleaning;
- no user/account identifiers, so user-level leakage cannot be checked;
- item ids are unique per outfit, so reuse of the same garment across outfits is only detectable through image hashing; different photos of the same product are not detected.

18. Data Leakage Prevention

- **Item-disjoint splits**: valid/test pairs containing an item, or a duplicate image of an item, seen in training are dropped (427 valid and 702 test pairs). `scripts/reproduce_results.py` recomputes the splits and reports item overlap train/valid, train/test and valid/test = **0 / 0 / 0**. The checkpoint records `test_items_seen_in_training = 0`.
- Duplicate images across splits are collapsed before splitting.
- The epoch is chosen on validation ROC-AUC and the decision threshold on validation F1; the ranker and the outfit threshold are fitted on validation data. The **test split is used only for the final report**.
- The demo wardrobe is drawn from the test split, so its items were never used for training or fitting.

- Known residual risk: the validation split is reused for several choices (epoch, threshold, ranker), so validation numbers are mildly optimistic; test numbers are not affected.

19. Training Methodology

```
python scripts/train_compatibility.py --polyvore-dir ... --metadata ... --images-dir ... \
--manifest ... --embeddings-dir ...
```

- Pairs: 90,695 observed positives in training, with an equal number of constructed negatives (half random cross-outfit pairs, half "hard" negatives that keep a real positive's category pairing).
- AdamW, lr 1e-3, weight decay 1e-4, batch 256, up to 15 epochs, early stopping with patience 3 on validation ROC-AUC, seed 42.
- Verification run: 7 epochs, best epoch 4, 55 s on the RTX 3050.
- The quality gate runs again before training and stops on INSUFFICIENT.

20. Evaluation Methodology

- Test split: 41,745 pairs (20,868 observed, 20,877 constructed), all items unseen in training.
- Metrics: ROC-AUC and PR-AUC with bootstrap 95% CIs (500 resamples), precision/recall/F1 at the validation-chosen threshold, Brier score, 10-bin expected calibration error, results on hard vs random negatives and per category pair.
- Paired bootstrap CI for the learned-minus-baseline ROC-AUC difference decides whether the model "beats the baseline".
- Ranker: pair ROC-AUC on the same test pairs; outfit-level TPR/FPR/AUC on real vs category-preserving fake test outfits.

21. Baselines

All baselines are scored on exactly the same test pairs:

- **Random**: uniform random scores.
- **CLIP cosine**: image-image cosine of the two items.
- **App heuristic**: the hand-written visual + category combination the app used before the learned model (`training/train._heuristic_scores`).
- **Fitted ranker without the learned model**: what a fresh clone (no checkpoint) uses.

22. Quantitative Results

Verified on 2026-10-04 by `scripts/reproduce_results.py` (re-scoring the saved checkpoint and the baselines with the current code on regenerated splits) and `scripts/fit_ranker.py`.

Item-disjoint test split, 41,745 pairs	ROC-AUC (95% CI)	PR-AUC	vs hard negatives only
Random	0.503 (0.498–0.509)	0.502	0.503
CLIP cosine	0.580 (0.575–0.586)	0.562	0.584
App heuristic	0.580 (0.575–0.586)	0.573	0.564
Learned model	0.725 (0.720–0.730)	0.714	0.716
Fitted ranker + learned	0.727	—	0.716
Fitted ranker, no learned model	0.604	—	0.594

- Learned minus CLIP: +0.145 ROC-AUC (paired bootstrap 95% CI 0.138–0.151).
- At the validation-chosen threshold (0.27): precision 0.58, recall 0.88, F1 0.70; Brier 0.226; ECE 0.11 (weak calibration).
- Per category pair the learned model ranges from 0.668 (one_piece+top) to 0.879 (bottom+bottom, 219 pairs).
- Combining signals adds almost nothing to the learned model at pair level (0.727 vs 0.725). The combiner's value is the validation-chosen outfit threshold and a fitted fallback when no checkpoint exists (0.604 vs CLIP 0.580).

How to read this. ROC-AUC is the probability that a random observed pair scores above a random constructed negative. **0.725 is not "72.5% accuracy"** and not "72.5% fashion understanding". These numbers measure agreement with Polyvore co-occurrence on this item-disjoint split; generalisation to users' photos was not measured. One training run, one seed.

23. CUDA/GPU Implementation

- `DEVICE=auto` resolves to CUDA when PyTorch reports it, else CPU; `cpu/cuda` force it. YOLO-World and CLIP are moved to the device at load time.
- The sidebar and Diagnostics show the active device, GPU name, PyTorch CUDA build, free VRAM and the device each model is on, and warn on a CPU fallback.
- **Installation trap (observed).** Ultralytics depends on torchvision; if pip resolves the newest torchvision from PyPI it can pull a newer CPU-only torch and silently replace the CUDA build. Installing `torch` and `torchvision` together from the CUDA index *first*, then the rest, avoids it.

Verified environment: Windows 11, Python 3.13, NVIDIA GeForce RTX 3050 6 GB Laptop GPU, PyTorch 2.11.0+cu128 (CUDA 12.8). CPU environment: PyTorch 2.14.1 CPU build on the same laptop. **No other GPU, CUDA version, Apple MPS, macOS or Linux setup was tested.**

CUDA verification (re-run 2026-10-04): CUDA available; YOLO-World and CLIP on `cuda:0`; 30 repeated detect + embed + generate cycles with allocated memory 719.2 MB → 719.2 MB (0.0 MB growth), peak 757 MB; models stay loaded across reruns; detector reload after restart 0.07 s; Diagnostics reports `active=cuda`, CUDA 12.8, no warning. The CPU fallback was exercised by the full test suite, the real-model tests and the smoke test in the CPU environment.

24. Performance Measurements

Single measurements on the verification laptop; your numbers will differ.

Operation	CUDA (RTX 3050)	CPU
Detector start (cached vocabulary)	0.36 s	~0.2 s (earlier run)
Detection, one photo (median of 10, incl. image load + crop saving)	29 ms	~190–220 ms (earlier run)
CLIP start	4.2 s	~5–7 s (earlier run)
CLIP embedding	4.6 ms / image (batch of 10)	not measured per image
Add one item (embed + colours + style + write)	46 ms	~90 ms (earlier run)
GPU memory, both models resident	719 MB allocated, 757 MB peak	—
Outfit generation, 29 items (k = 3...20, warm)	16–58 ms	26–63 ms
Outfit generation, 185 synthetic items (72,450 valid candidates)	—	0.8 s (k=5), 2.0 s (k=20)

25. UI/UX

Four Streamlit pages (`app.py` is the only entry point):

1. **Wardrobe:** upload → *Detect clothing* → review (alternatives, flags, suggestions) → *Add selected*; gallery of **every** item with per-category counts, filter (incl. *Needs review*), search, sort, display-only pagination, confirm / edit / delete, and a guarded *Clear wardrobe*.
2. **Outfit Generator:** style, occasion, must-include item, number of looks (1–20), outerwear/accessory toggles. Summary "Showing N of M strong outfit candidates · X valid combinations scored from Y usable items"; excluded items with reasons; each look shows images, ✓ reasons, ⚠ caveats and a *Score details* panel; a fewer-than-requested message instead of padding; the pipeline funnel.
3. **Item Recommendations:** every usable compatible partner for a chosen item, ranked, with the strongest signals.
4. **System / Diagnostics:** device and model state, learned-model metrics, the fitted ranker summary (including the unused colour signal), the last outfit run's funnel and exposure table, embedding and file health, repair actions.

26. Diagnostics

The outfit funnel is computed by the real pipeline, not reconstructed: wardrobe items → eligible (excluded by reason) → eligible per category → structural combinations (incl. optional layers) → rejected by validity → valid candidates scored → pruned for compute → above threshold → returned → unique items in results → time; plus per-item exposure. Diagnostics also reports missing/corrupt/stale embeddings, missing images, unknown categories and unreferenced files (removable; referenced data is never deleted automatically).

27. Testing

Suite	Command	Result (2026-10-04)
Offline suite (fakes replace model inference only)	<code>pytest</code>	167 passed, 4 skipped (the opt-in tests), CPU and CUDA
Opt-in real models	<code>RUN_MODEL_TESTS=1 pytest tests/test_real_models.py tests/test_real_ui_workflow.py</code>	4 passed (CPU and CUDA)
End-to-end smoke	<code>python scripts/smoke_test.py</code>	passed (CPU, CUDA, and CUDA environment forced to CPU)
Lint / types	<code>ruff check ., ruff format --check ., mypy src scripts app.py</code>	clean

The offline suite runs the real code for detector post-processing, region grouping and flags, storage, review state, validity, generation, diversity, the training loop, the dataset audit and download helpers, and all four Streamlit pages (`streamlit.testing`). Notable tests: the adversarial validity set; the bus-like jumpsuit chain (flagged → excluded → corrected → valid outfits); 29-item full coverage (every core item reaches candidate generation, no truncation, unique items grow with k); no-padding and threshold tests; compute-guard pruning reporting; explanations never mention uncomputed attributes; the opt-in real bus.jpg check.

28. Browser Verification

A visible-browser run (Chrome, CUDA backend) on the 29-item wardrobe:

- gallery: 29 items, per-category counts, pagination "Page (of 2)";
- bus.jpg upload: 10 regions, 4 needing review; the jumpsuit unticked with "Also detected as: coat (Outerwear) 0.20, jacket (Outerwear) 0.13" and *Outerwear* preselected; ticked and added → stored as confirmed outerwear; wardrobe 36 items;
- 5 looks: "Showing 5 of 406 strong outfit candidate(s) · 469 valid combinations scored from 36 usable item(s)"; 10 looks: 10 distinct, valid, varied looks;
- must-include gown, 20 requested: 3 shown with "3 strong outfit(s) found from your current wardrobe (you asked for 20)";
- score details, funnel, Diagnostics and Item Recommendations rendered; no browser console errors.

The run found and fixed one bug (flag messages listed alternatives twice). Screenshots are not committed because they contain Polyvore product images.

29. Failure Cases

- Zero-shot detection mislabels (person region as "jumpsuit", suit as "jumpsuit", pants vs jeans) and misses small accessories.
- Confidently wrong labels without competing evidence are not flagged.
- Rectangular crops include background and skin, which affect colours, style estimates and embeddings.
- Noisy dataset categories propagate into anything built from Polyvore (the "tank top" that is a pair of sunglasses).

- The two Ultralytics sample photos contain no detectable top, so they produce no outfit: the app explains what is missing rather than inventing one.

30. Fixes Implemented

See `docs/CHANGELOG_FINAL.md` for the chronological list. In short: a working single entry point; one detector API; one CLIP loader; SQLite storage with atomic writes and integrity checks; removal of fake training and fake attributes; real dataset acquisition with verification and a quality gate; an item-disjoint learned model that beats its baselines; CUDA support with diagnostics; detector uncertainty and review; a hard validity gate; full-wardrobe generation; diversity; a validation-fitted ranker; removal of colour from ranking; an outfit funnel; browser QA.

31. Known Limitations

- Detection is zero-shot and imperfect; review is required.
- The ranker and threshold were fitted on Polyvore product images; crops from users' photos differ (domain shift) and scores are not recalibrated for them. Generalisation to users' photos was not measured.
- The learned model and ranker are uncalibrated ranking signals.
- Co-occurrence in curated sets $\neq$ human preference; random negatives are easy.
- Style is a coarse CLIP zero-shot estimate; the occasion table is hand-written.
- Colour names are coarse (17 names) and lighting-sensitive; colour is not used for ranking.
- Material, pattern, fit and silhouette are not inferred.
- Duplicate detection catches re-added images, not the same garment photographed again.
- Single user, local storage, no authentication. Exhaustive enumeration suits personal wardrobes; above 200,000 combinations per template, pools are pruned.
- One training run, one seed; validation reused for several choices.
- Verified only on Windows 11 with one NVIDIA GPU and a CPU-only setup.
- Dataset images are research-use only and are not redistributed. The code is MIT-licensed (`LICENSE`); the licence does not extend to the dataset or the third-party model weights.

32. Reproducibility

Exact commands are in `docs/REPRODUCIBILITY.md`. The committed records are:

- `artifacts/dataset_manifest.json`: sources, pinned revisions, sizes, SHA-256;
- `artifacts/dataset_audit.{json,md}`: the audit and gate verdict;
- `src/recommendation/ranker_model.json`: fitted ranker weights, dropped features with reasons, thresholds, validation/test metrics;
- `scripts/reproduce_results.py`: re-derives the learned-model and baseline numbers from the checkpoint and regenerated splits;
- `scripts/demo_wardrobe.py`: rebuilds the 29-item test wardrobe.

Model weights, the dataset and wardrobes are not committed.

33. Installation

```
python -m venv .venv
.venv\Scripts\activate          # macOS/Linux: source .venv/bin/activate
python -m pip install --upgrade pip
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128 # CUDA (first!)
# or: pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements-dev.txt
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

Requires Python ≥ 3.10 (verified with 3.13), `git` on PATH (one dependency installs from GitHub), ~1.5 GB for model weights and ~6 GB more for the optional dataset.

34. Running the Application

```
streamlit run app.py          # http://localhost:8501
```

Weights download on first use. The wardrobe lives in `DATA_DIR` (default `data/`), so a fresh clone starts with an empty wardrobe. To try the app on the 29-item test wardrobe instead (requires the downloaded dataset):

```
python scripts/demo_wardrobe.py --data-dir data/demo29
DATA_DIR=data/demo29 streamlit run app.py      # PowerShell: $env:DATA_DIR="data/demo29"; streamlit run app.py
```

35. Training the Compatibility Model

```
python scripts/download_dataset.py
python scripts/embed_dataset.py --images-dir data/datasets/maryland-polyvore/prepared/images \
  --out data/datasets/maryland-polyvore/embeddings
python scripts/train_compatibility.py --polyvore-dir data/datasets/maryland-polyvore/prepared \
  --metadata data/datasets/maryland-polyvore/prepared/metadata.json \
  --images-dir data/datasets/maryland-polyvore/prepared/images \
  --manifest data/datasets/maryland-polyvore/manifest.json \
  --embeddings-dir data/datasets/maryland-polyvore/embeddings
python scripts/fit_ranker.py
python scripts/reproduce_results.py
```

36. Future Work

- Measure performance on real user photos with human judgements (the main missing evaluation).
- Segmentation masks instead of rectangular crops.
- Fine-tune or calibrate detection on fashion data; flag more error patterns.
- A data-driven colour signal (e.g. colour-pair statistics learned from training outfits) to replace the rejected hand-written rules.
- Harder negatives and multiple seeds for the learned model; calibration.
- Outfit-level (not pairwise) compatibility models.
- Testing on Linux/macOS and other GPUs.

37. Final Assessment

The application does what it claims and no more. It turns photos into a reviewable wardrobe, refuses to recommend structurally impossible outfits, uses the whole wardrobe, explains its choices from computed values and is explicit about uncertainty. The learned compatibility signal clearly beats simple baselines on a Polyvore-derived item-disjoint evaluation (0.725 vs 0.580 ROC-AUC), and the ranking and threshold are fitted on validation data rather than set by hand. One intuitive component, the colour rules, failed that test and was removed from ranking.

What is not established: quality on users' own photos, agreement with human taste, and behaviour on other platforms. The system should be read as a carefully engineered, honestly evaluated prototype, not a validated fashion judge.